Universidad Francisco de Vitoria

UFV Madrid

SecureBank

Sistema Bancario Concurrente

Memoria técnica

GRADO EN INGENIERÍA INFORMÁTICA

Luka Rojas Rios

Sistemas Operativos · Curso 2025-2026

Índice

1. Descripción general
2. Arquitectura del sistema
3. Estructuras de datos
4. Sincronización
5. Flujo de una operación completa
6. Detección de anomalías
7. Fichero de configuración
8. Plan de pruebas
9. Conclusiones
10. Referencias

1. Descripción general

SecureBank simula un banco con varios usuarios operando a la vez. El sistema está formado por tres ejecutables (`banco`, `usuario`, `monitor`) que se comunican entre sí con colas de mensajes System V, tuberías con nombre (FIFOs) y semáforos POSIX nombrados. Cada sesión de usuario tiene su propia ventana de terminal, las cuentas viven en un fichero binario compartido y todas las operaciones quedan registradas en un log.

Utiliza un multi-proceso porque es la forma más clara de separar responsabilidades. El proceso `banco` gestiona el alta de sesiones y escribe el log. Cada proceso `usuario` maneja el menú interactivo y las operaciones de una cuenta concreta. El proceso `monitor` vigila en paralelo y detecta patrones sospechosos. Si todo estuviera en un mismo proceso con hilos, compartir memoria sería más fácil pero perdería el valor pedagógico del proyecto: trabajar con IPC real entre procesos separados.

Las operaciones dentro de cada sesión sí usan hilos (`pthread`) porque aísla cada operación. Uso `pthread_join` justo después del `pthread_create` para serializar la entrada del usuario y evitar que dos operaciones del mismo menú compitan por el mismo semáforo a la vez.

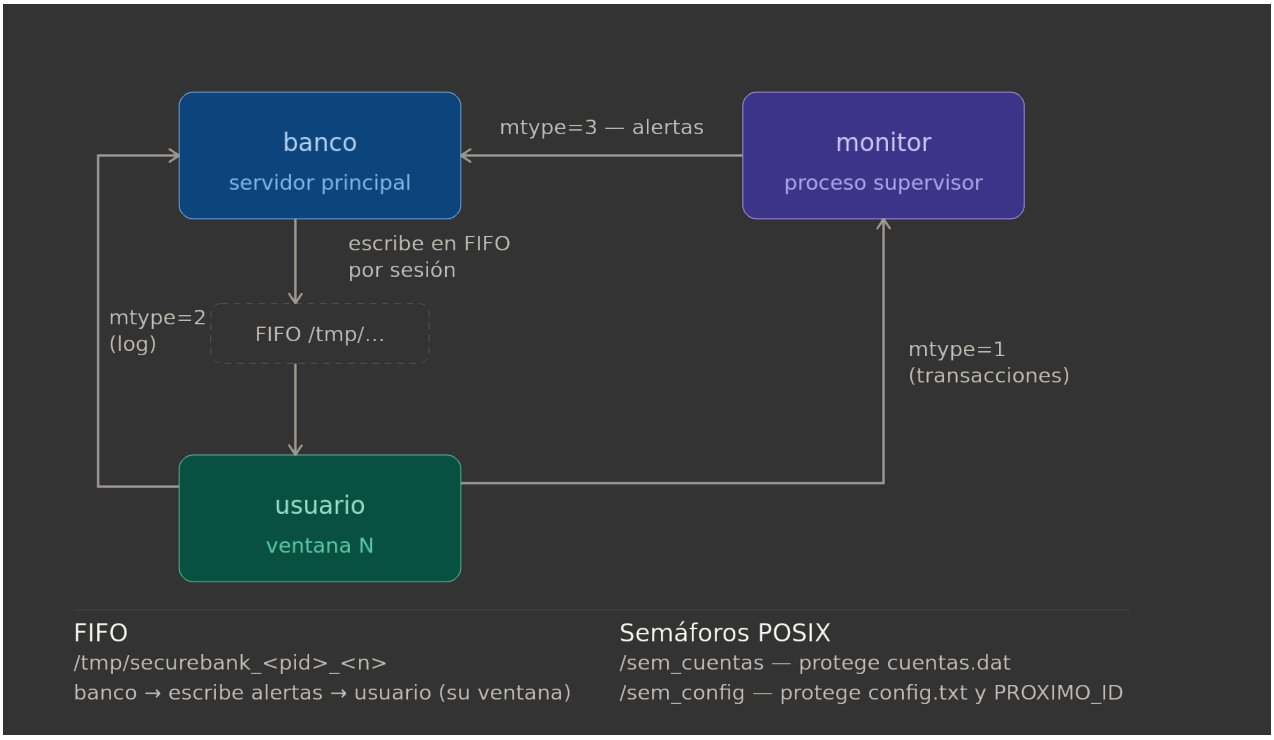
2. Arquitectura del sistema

El sistema tiene tres procesos. El principal es `banco`: arranca, lee la configuración, abre los semáforos, crea la cola de mensajes y lanza `monitor` con `fork` + `execv`. Después entra en su bucle principal, pide un número de cuenta y, por cada sesión que se abre, lanza una ventana nueva con `gnome-terminal --wait` ejecutando `./usuario`.

El proceso `monitor` vive durante toda la ejecución. Solo termina cuando `banco` le manda SIGTERM al cerrar. Lee los mensajes de tipo 1 que le llegan de los hilos de `usuario` y analiza cada transacción.

Cada proceso `usuario` es el más interactivo. Muestra un menú, lanza un hilo por cada operación, escribe o lee `cuentas.dat` protegido por el semáforo `/sem_cuentas`, y envía mensajes a la cola: tipo 1 al monitor y tipo 2 a `banco` para el log.

Diagrama de conexiones. El campo `mtype` indica el tipo de mensaje dentro de la cola compartida, no un canal físico separado:



Una sesión cualquiera queda así: `banco` crea una FIFO única, abre su extremo de escritura con `O_RDWR`, lanza `gnome-terminal --wait -- ./usuario <cuenta> <msgid> <fifo>` con `fork`+`execvp`, y vuelve al bucle. Cuando el usuario sale (opción 6 o alerta con BLOQUEO), el `usuario` termina, la ventana se cierra, y `banco` reaparea el pid con `waitpid(WNOHANG)` en su

siguiente iteración del bucle principal.

3. Estructuras de datos

Las cuatro estructuras principales están en `estructuras.h`.

La estructura `Cuenta` almacena el número de cuenta, el titular (hasta 49 caracteres), los tres saldos en EUR, USD y GBP, y un contador de operaciones. Cada registro ocupa exactamente `sizeof(Cuenta)` bytes dentro de `cuentas.dat`. En mi máquina son 72 bytes: un int de 4, 50 bytes de char, 3 floats de 4, otro int de 4, más padding para alinear a 4 bytes.

La estructura `DatosMonitor` lleva lo que el monitor necesita para analizar una operación: cuenta origen, cuenta destino, tipo de operación, cantidad, divisa y timestamp.

La estructura `DatosLog` lleva lo que `banco` necesita para registrar una línea en el log: id de cuenta, pid del hijo, tipo de operación, cantidad, divisa, estado y timestamp.

La estructura `msgbuf` es la plantilla para `msgsnd` y `msgrcv`. Tiene un `long mtype` y una `union` con los dos tipos de mensaje anteriores. La unión ahorra espacio y unifica el envío: el mismo buffer vale para los tres canales lógicos del sistema. Eso obliga a pasar `sizeof(m.info)` como tamaño al `msgrcv`, no `sizeof(m)` que incluiría el mtype, y tampoco `sizeof(DatosMonitor)` o `sizeof(DatosLog)` por separado porque el otro canal quedaría truncado. Este detalle lo tuve claro desde el principio porque viene en el esqueleto.

El fichero binario `cuentas.dat` funciona por registros de tamaño fijo. Cada cuenta nueva se añade al final con `fseek(SEEK_END)` + `fwrite`. Para leer o modificar una cuenta concreta hay que saber en qué posición está. El enunciado propone calcular el offset como `(numero_cuenta - ID_INICIAL) * sizeof(Cuenta)`, pero esa fórmula asume que las cuentas son consecutivas empezando en 1001 y sin huecos. Eso me dio problemas (lo cuento en las conclusiones) y al final hago una búsqueda lineal por `numero_cuenta` dentro del fichero mediante la función `find_record_index` en `usuario.c`. Es más lento pero no depende de un invariante frágil.

4. Sincronización

Uso dos semáforos POSIX nombrados.

El semáforo `/sem_cuentas` protege el acceso a `cuentas.dat`. Lo adquieren los hilos de `usuario` antes de cualquier lectura o escritura, y también `banco` cuando crea una cuenta o lista las cuentas. Así nunca hay dos escrituras pisándose en mitad de un `fwrite`.

El semáforo `/sem_config` protege `config.txt`, en particular la lectura y escritura de `PROXIMO_ID`. Solo lo toca `banco` en `crear_cuenta` porque es el único que cambia ese parámetro, pero tenerlo nombrado permite que otros procesos lo usen en el futuro sin tocar la arquitectura.

La razón de usar `sem_open` en vez de `sem_init` es que `sem_init` crea un semáforo en la memoria del proceso que lo llama, y solo sirve entre hilos del mismo proceso o entre procesos si lo pones en memoria compartida con `shm_open` o `mmap(MAP_SHARED)`. `sem_open` crea un semáforo con un nombre en el kernel (aparece como fichero en `/dev/shm/sem.sem_cuentas`) accesible por cualquier proceso que conozca el nombre, sin necesidad de memoria compartida. Como aquí `banco` y los múltiples `usuario` son procesos distintos sin relación directa de memoria, los nombrados son la opción más directa.

Orden de adquisición. Cuando hacen falta ambos, que solo pasa en `crear_cuenta`, tomo primero `/sem_config` y después `/sem_cuentas`. Siempre el mismo orden. Si en algún momento añadiera código que los tomara en orden contrario habría riesgo de interbloqueo, pero con un único punto que toma los dos no hay problema en la práctica.

El proceso `usuario` tiene un problema distinto de sincronización. Tiene que atender al teclado y a la FIFO de alertas al mismo tiempo. Si hiciera `scanf` sin más quedaría bloqueado hasta que el usuario escriba, y las alertas del banco no se verían a tiempo. La solución es `poll()` sobre dos descriptores: `STDIN_FILENO` y el fd de la FIFO abierta en lectura. `poll` duerme al proceso hasta que alguno tiene datos, y luego yo miro `revents` para saber cuál. Así la sesión se queda dormida sin consumir CPU y reacciona a cualquiera de los dos canales.

5. Flujo de una operación completa

Un depósito de 500 EUR en la cuenta 1001, paso a paso, desde que el usuario pulsa 1 hasta que la línea aparece en `transacciones.log`:

11. El usuario escribe `1001` en la ventana de `banco`. Tras el `poll`+`scanf`, `banco` verifica con `buscar_cuenta` que existe y llama a `launch_session`.
12. `launch_session` crea una FIFO en `/tmp/securebank_<pid>_<n>`, la abre con `O_RDWR` (así el extremo de escritura nunca bloquea ni se cierra accidentalmente), y lanza `gnome-terminal --wait --working-directory=<cwd> -- ./usuario 1001 <msgid> <fifo>` con `fork`+`execvp`.
13. Aparece una ventana nueva. Dentro, `usuario` lee la configuración, abre los dos semáforos con `sem_open(O_CREAT)`, abre la FIFO en modo solo lectura, limpia la pantalla y muestra el menú.
14. El usuario pulsa `1`. El `poll` devuelve con POLLIN en stdin. `scanf` lee la opción, la pantalla se limpia, se imprime la cabecera “Deposit” y se piden importe (500) y divisa (0 para EUR).
15. `procesar_opcion` reserva un `DadosOp` en memoria con todos los datos de la operación, crea un hilo con `pthread_create(ejecutar_operacion, op)` y hace `pthread_join` justo después para esperar al resultado.
16. El hilo pide `sem_wait(sem_cuentas)`, abre `cuentas.dat` en modo `rb+`, busca el registro de 1001 con `find_record_index`, hace `fseek` al offset correspondiente, hace `fread` para cargar la estructura `Cuenta`, suma 500 al saldo EUR, incrementa `num_transacciones`, vuelve a hacer `fseek` al mismo offset y hace `fwrite` para guardar. Cierra el fichero y hace `sem_post`.
17. El hilo construye un `DatosMonitor` con los datos de la operación y lo envía a la cola con `mtype=1` y `msgsnd`. Después construye un `DatosLog` y lo envía con `mtype=2`. Libera la memoria del `DadosOp` y termina.
18. El hilo principal de `usuario` vuelve del `pthread_join`, muestra el menú de nuevo y se queda esperando en `poll`.
19. El proceso `monitor`, que estaba bloqueado en `msgrcv(mtype=1, 0)`, recibe el mensaje. `analizar_transaccion` evalúa las tres reglas. Como un depósito normal no dispara ninguna, solo lo guarda en el histórico circular y vuelve a `msgrcv`.
20. El proceso `banco`, en su bucle, hace `msgrcv(mtype=2, IPC_NOWAIT)` y recibe el `DatosLog`. `format_and_log` compone la cadena `Deposit account 1001: +500.00 EUR` y la añade a `transacciones.log` con un timestamp delante.
21. `banco` continúa con `msgrcv(mtype=3, IPC_NOWAIT)` (no hay alertas), `waitpid(-1,`

`WNOHANG)` (el usuario sigue vivo) y vuelve al `poll` de stdin. Todo el ciclo desde el `pthread_create` hasta que aparece la línea en el log dura unos pocos milisegundos.

6. Detección de anomalías

El proceso `monitor` guarda un buffer circular con las últimas 50 transacciones y un contador por cuenta de retiros grandes consecutivos. El análisis se ejecuta en `analizar_transaccion`, que se llama dentro del bucle tras cada `msgrcv(mtype=1)`.

La primera regla detecta retiros grandes consecutivos. Si el tipo de operación es retiro (código 2) y la cantidad supera el 80 % del `LIM_RET_*` correspondiente a la divisa, incrementa el contador de esa cuenta. Si el contador llega al valor de `UMBRAL_RETIROS` (por defecto 3), el monitor dispara alerta con el motivo `consec withdrawals` y resetea el contador. Cualquier otra operación de esa cuenta también resetea el contador, para que “consecutivos” signifique de verdad seguidos.

La segunda regla detecta transferencias repetidas entre el mismo par de cuentas. Si el tipo es transferencia (código 3), el monitor recorre el histórico contando cuántas veces aparece el mismo par `(origen, destino)`. Si con la nueva suman `UMBRAL_TRANSFERENCIAS` o más, dispara alerta `repeat transfers`.

La tercera regla detecta accesos simultáneos desde pids distintos. Para operaciones que no son transferencia, el hilo de `usuario` mete su pid en el campo `cuenta_destino` del `DatosMonitor`. Esto es un apaño necesario porque la estructura no tiene un campo pid propio y no puedo modificar la cabecera congelada. El monitor mira las últimas 5 entradas del mismo `cuenta_origen` y, si encuentra otro pid distinto, dispara alerta `concurrent access`. La regla no se aplica a transferencias porque ahí `cuenta_destino` sí es una cuenta real.

Flujo completo de una alerta. El monitor construye un `DatosLog` con `tipo_op=-1`, mete el motivo (máximo 19 caracteres) en el campo `timestamp` de la estructura (otro apaño por la cabecera congelada) y lo envía con `mtype=3`. El proceso `banco` lo recibe en su bucle y lo maneja en `format_and_log_alert`: escribe `[ts] ALERT account X: <motivo>` en el log y, si la cuenta tiene una sesión abierta en el array `hijos[]`, escribe `BLOQUEO: <motivo>` en la FIFO correspondiente con un `write()`. El proceso `usuario` lo ve por `poll` con POLLIN en la FIFO, lo imprime como `[BANK ALERT]: ...`, y si detecta la palabra `BLOQUEO` llama a `exit(0)` tras una pausa de dos segundos para que al usuario le dé tiempo a leer el mensaje antes de que se cierre la ventana.

7. Fichero de configuración

El fichero `config.txt` es texto plano con una pareja `clave=valor` por línea. Las líneas que empiezan por `#` o están vacías se ignoran. Los tres ejecutables lo leen una vez al arrancar con una función `leer_config` muy sencilla: `fgets` línea a línea, `sscanf(..., "%63[^=]=%127s", key, val)` para partir, y un encadenado de `strcmp` que rellena una estructura `Config` global.

No hay recarga en caliente. Si cambio `config.txt` con el sistema corriendo, los procesos existentes no se enteran. Para que surta efecto hay que reiniciar `banco`. Lo dejé así porque no lo pedía el enunciado y añadir recarga complicaría bastante, especialmente sincronizar la relectura entre procesos sin introducir lecturas parciales.

Los parámetros se agrupan por uso. El parámetro `PROXIMO_ID` lo usa `banco` para asignar ids de cuenta. Los `LIM_RET_*` y `LIM_TRF_*` los usa `usuario` para rechazar operaciones que pasen del límite, y el monitor los usa como referencia para decidir qué cuenta como retiro grande. Los valores `UMBRAL_RETIROS` y `UMBRAL_TRANSFERENCIAS` los usa el monitor para decidir cuándo dispara alerta. El parámetro `NUM_HILOS` está declarado pero no lo uso activamente: cada operación arranca su propio hilo y hace `join` inmediato, así que nunca hay más de uno activo por sesión. Los parámetros `ARCHIVO_CUENTAS` y `ARCHIVO_LOG` son las rutas a los ficheros de datos y log. Los valores `CAMBIO_USD` y `CAMBIO_GBP` los usa `usuario` para las conversiones de divisa y el cálculo del total en EUR.

La actualización de `PROXIMO_ID` tras crear una cuenta la hace `banco` con la función `reescribir_config`: lee `config.txt` línea a línea y copia cada línea a `config.txt.tmp`, reemplazando la que empieza por `PROXIMO_ID=` por el nuevo valor. Al final hace `rename` para sustituir el fichero original de forma atómica. Esto evita que un crash a mitad de escritura deje el fichero corrupto.

8. Plan de pruebas

Prueba a. Crear cuenta, depositar 500 EUR e inspeccionar cuentas.dat.

Ejecuté `make distclean && make && ./banco`. Introduje `0` y como titular `alice`, y el sistema creó la cuenta 1001. Abrí la sesión de 1001 desde el mismo banco y desde su ventana hice un depósito de 500 en EUR. La línea `Deposit ok. New balance: 500.00 EUR` apareció en la ventana de `usuario`, y la línea `[timestamp] Deposit account 1001: +500.00 EUR` en `transacciones.log`. Después hice `hexdump -C cuentas.dat | head`. Los primeros 4 bytes eran `e9 03 00 00` que en little-endian es `0x03E9 = 1001`. Detrás estaba `alice` en ASCII, y en los offsets correspondientes al saldo EUR estaba el float 500.0 (`00 00 fa 43`). Confirmado: el registro queda bien escrito en disco.

Prueba b. Retiro superior al saldo.

Con la cuenta 1001 a 500 EUR, intenté retirar 1000 EUR. La operación imprimió `Insufficient funds`, no modificó el fichero (lo comprobé volviendo a consultar el saldo: seguía a 500) y en el log apareció `Withdrawal account 1001: -1000.00 EUR (failed)`. El hilo de la operación detecta el saldo insuficiente antes de modificar nada y envía el `DatosLog` con `estado=0`, que `format_and_log` traduce al sufijo `(failed)`.

Prueba c. Retiro superior al LIM_RET_EUR.

Deposité 20000 EUR. Con `LIM_RET_EUR=5000` intenté retirar 6000. Respondió `Amount exceeds withdrawal limit (5000.00)`. No hubo cambios en el saldo. En el log quedó con `(failed)`. Este caso y el anterior los comprueba el propio hilo de `usuario` antes de escribir; no hace falta que el monitor intervenga.

Prueba d. Retiros consecutivos para disparar la alerta del monitor.

Bajé `LIM_RET_EUR` a 100 y dejé `UMBRAL_RETIROS` a 3. Reinicié el sistema (recordar que la config se lee al arrancar). Deposité 10000 EUR en 1001. Hice tres retiros de 100 EUR seguidos. Los dos primeros pasaron como retiros normales. Al terminar el tercero, en la ventana de `usuario` apareció `[BANK ALERT]: BLOQUEO: consec withdrawals`, y dos segundos después la ventana se cerró sola. En `transacciones.log` se ven los tres retiros seguidos de la línea `ALERT account 1001: consec withdrawals`. La primera vez que probé esto me pareció que el contador no se reseteaba entre sesiones, pero revisando el código vi que el monitor mantiene el histórico en memoria durante toda la vida del sistema, y eso es intencionado: si un atacante abre y cierra sesiones para “blanquear” el contador, el monitor no debería olvidarse. Lo dejé así porque cumple con lo que el enunciado pide sobre detección en tiempo real.

Prueba e. Dos sesiones concurrentes con transferencia entre ellas.

Con cuentas 1001 y 1002 ya creadas, desde el banco escribí `1001` y pulsé Enter, y a continuación `1002`. Aparecieron dos ventanas de `gnome-terminal` simultáneamente, cada una con su menú. En 1001 deposité 1000 EUR. En 1002 consulté el saldo inicial (0). Transferí 300

EUR de 1001 a 1002 desde la ventana de 1001. Ambas ventanas siguieron respondiendo: consulté el saldo de 1001 (700 EUR) y el de 1002 (300 EUR). La línea `Transfer 1001 -> 1002: 300.00 EUR` apareció correctamente en el log. Este test me dio trabajo antes de llegar a la versión final: con pipes anónimas heredadas por `execv` la sesión se ejecutaba en la misma ventana que `banco` y bloqueaba nuevas entradas. Cambiar a FIFOs con nombre más `gnome-terminal --wait` lo resolvió. También tuve que pasar `--working-directory=<cwd>` porque el servidor de gnome-terminal abría la ventana con cwd en `$HOME` y `fopen("cuentas.dat")` fallaba silenciosamente.

Prueba f. Cierre con Ctrl+C y comprobación de recursos del sistema.

Con el sistema corriendo (una sesión abierta, monitor activo), pulsé Ctrl+C en la ventana de `banco`. El manejador `handle_sigint` mata al monitor con SIGTERM, mata las sesiones abiertas también con SIGTERM, cierra los semáforos, hace `sem_unlink` de ambos, `msgctl(IPC_RMID)` de la cola y sale. Después ejecuté `ipcs -q` (no aparecía ninguna cola a mi nombre), `ls /dev/shm/sem.sem_cuentas /dev/shm/sem.sem_config` (`No such file or directory` en los dos) y `pgrep monitor` (vacío). La ventana de `usuario` que había sobrevivía al `banco` durante un instante pero moría inmediatamente al recibir SIGTERM del `cleanup`. Todo limpio.

9. Conclusiones

El proyecto funciona. Varias sesiones corriendo en paralelo en ventanas separadas, cuentas persistentes, log completo con distinción entre operaciones normales y alertas, y un cierre que no deja semáforos ni colas colgando. El menú interactivo quedó razonablemente usable después de añadir limpieza de pantalla entre operaciones para que no se acumulara el scroll.

Lo que más trabajo me dio fueron dos cosas. La primera, el paso de la tubería al hijo a través de `execv`. Al principio usaba pipes anónimas y funcionaba, pero la sesión corría en la misma terminal que `banco` y bloqueaba la entrada de nuevos usuarios, que era justo lo que el proyecto quería evitar. Cambié a FIFOs nombradas en `/tmp` para que `usuario` pudiera abrirlas por ruta después de lanzarse en un terminal nuevo. Junto con `gnome-terminal --wait` para que `banco` pudiera seguir reaparando al hijo con `waitpid`, aquello resolvió el problema del multi-sesión.

La segunda, el cálculo de offsets en `cuentas.dat`. La fórmula `(numero_cuenta - ID_INICIAL) * sizeof(Cuenta)` que sugiere el enunciado asume que los ids son consecutivos desde 1001. Estuve un rato depurando un bug en el que cualquier operación respondía `Cannot read account record` sin saber por qué. Al final resultó que durante las pruebas había editado `PROXIMO_ID` a mano, así que existía la cuenta 1004 pero el fichero solo tenía un registro al offset 0 y el código intentaba leer al offset 216. Lo arreglé con búsqueda lineal en `find_record_index`. Es peor en rendimiento (O(n) por operación) pero no depende de que el usuario no toque el fichero de configuración. Para un banco con miles de cuentas habría que construir un índice en memoria al arrancar, pero para este alcance la búsqueda lineal sobra.

También añadí un auto-reseteo: cuando `banco` detecta que `cuentas.dat` está vacío al arrancar, devuelve `PROXIMO_ID` a 1001 y reescribe `config.txt`. Esto cubre `make distclean` y cualquier borrado manual del fichero, y evita el tipo de desincronización que me comió media tarde.

Cosas que mejoraría con más tiempo: usar un índice en memoria para las cuentas en vez de búsqueda lineal en cada operación; añadir un campo `bloqueada` a la estructura `Cuenta` para que el bloqueo sea persistente (ahora mismo un usuario bloqueado puede reconectar); y detectar qué emulador de terminal está disponible para no depender exclusivamente de `gnome-terminal`.

10. Referencias

Páginas man de Linux consultadas durante el desarrollo: `fork(2)`, `execv(3)`, `execvp(3)`, `msgget(2)`, `msgsnd(2)`, `msgrcv(2)`, `msgctl(2)`, `sem_open(3)`, `sem_wait(3)`, `sem_post(3)`, `sem_unlink(3)`, `pthread_create(3)`, `pthread_join(3)`, `poll(2)`, `fseek(3)`, `fread(3)`, `fwrite(3)`, `pipe(2)`, `mkfifo(3)`, `waitpid(2)`, `signal(7)`, `ipcs(1)`.

Material de la asignatura Sistemas Operativos, Grado en Ingeniería Informática (apuntes y ejercicios de los temas 1 a 4: procesos, concurrencia, sincronización, comunicación entre

procesos).

Enunciado del proyecto: “Proyecto integrador, Temas 1 - 4 SSOO: Banco Parte 1”, documento facilitado por el profesor.